Del AVR al ESP32

El ESP32 no tiene DDRB, PORTB ni PINB, pero **la idea es la misma**: un pin se controla escribiendo bits en **registros mapeados en memoria**. Cambian los nombres y el ancho (32 bits).

Hardware · Software · código · Regla · decisión · Error · ojo · ◇ decisión

1 Leer-modificar-escribir: se pierde un cambio

PORTB |= (1 << PB5) son 3 pasos

1	`loop()` lee (`in`)	copia el registro	`copia = 0010 0000`
2	**Interrupción**	cambia el registro **entre medio**	`PORTB → 0010 0001`
3	`loop()` calcula (`eor`)	con la copia **vieja**	`resultado 0000 0000`
4	`loop()` escribe (`out`)	el resultado viejo	`PORTB ← 0000 0000`
5	**El cambio se perdió**	el de la interrupción	

2 W1TS y W1TC: escritura atómica

una sola escritura, sin leer antes

W1TS
Write 1 to set: los bits en 1 se **encienden**. Los bits en 0 no se tocan.

W1TC
Write 1 to clear: los bits en 1 se **apagan**. Los bits en 0 no se tocan.

```
GPIO_OUT antes    0 1 0 1
escribo en W1TS   0 0 1 0
GPIO_OUT después  0 1 1 1
```

BLINK CON REGISTROS DEL ESP32

```
#include "soc/gpio_reg.h"
constexpr uint8_t PIN_LED = 16;
constexpr uint32_t MASK_LED = 1UL << PIN_LED;

void setup() {
  pinMode(PIN_LED, OUTPUT);                 // configura
}
void loop() {
  REG_WRITE(GPIO_OUT_W1TS_REG, MASK_LED);   // como PORTB |=
  delay(500);
  REG_WRITE(GPIO_OUT_W1TC_REG, MASK_LED);   // como PORTB &= ~
  delay(500);
}
```

3 De digitalWrite al pin

arriba, más fácil · abajo, más control

1. **Tu código** — `digitalWrite(16, HIGH)`
2. **Core Arduino-ESP32** — llama a ESP-IDF
3. **ESP-IDF (Espressif)** — `gpio_set_level(16, 1)`
4. **Registro** — `GPIO_OUT_W1TS_REG ← 1 << 16`
5. **Hardware** — el pin 16 sube a **3,3 V**

4 Del reset al `loop()`

1. **Reset** — arranca con la alimentación
2. **Bootloader** — lee los pines de **strapping**
3. `app_main()` — ESP-IDF
4. `setup()` — configura **una vez**
5. `loop()` — se repite **siempre**

`loop() se repite: leer entradas → decidir → escribir salidas`

5 Variables local, global y static

valor en la 3.ª vuelta si cada una se incrementa en cada loop()

Global
Se crea **una vez** y conserva su valor.
`global=3`

Local
Se crea **de nuevo** en cada vuelta.
`local=1`

Static
Se crea **una vez** y conserva su valor.
`static=3`

6 Mapa de pines del ESP32 DevKit V1

verde · amarillo · rojo

Seguros usar estos

16 17 18 19 21 22 23 25 26 27 32 33

Para LEDs y pulsadores. En la AP2: LEDs en 16, 17, 18, 19 y pulsador en 21.

Evitar strapping

0 2 5 12 15

El chip los lee al arrancar: **no conectar entradas**. GPIO0 en bajo = modo de carga (botón BOOT). **GPIO12 en alto: la placa no arranca**.

Con cuidado solo entrada

34 35 36 VP 39 VN

Solo lectura y **sin pull-up interna**: usar pull-up externa (10 kΩ).

Prohibido no usar

1 TX0 3 RX0 6–11 flash

UART0 carga el programa y el Monitor Serie. 6–11 son la flash.

7 Límites eléctricos

los pines trabajan a 3,3 V

Tensión

No toleran 5 V: un sensor de 5 V directo puede dañar el pin.

Corriente

Máximo **10 mA** por pin.

Resistencia de un LED

`R = (3,3 V − Vf) / I`

SI EL PROGRAMA NO SUBE

Mantener el botón **BOOT** mientras aparece «Connecting...».

WOKWI NO REPRODUCE ESTAS FALLAS

Los problemas de strapping solo aparecen con la **placa real**.